NLP 课程小组作业 · 第一次个人汇报 · 题目九

From Code Assistants to Code Guardians: Real-Time Hallucination Detection for AI-Generated Code in CI/CD Pipelines

相关文献综述：代码幻觉的分类、检测与 CI/CD 集成

小组成员：李甘、张星烁、连瑾昭
汇报人：李甘

题目来源： openi.pcl.ac.cn/luckh2/nlp26projects
仓库： github.com/Nictheboy/college-nlp-lab

起点：一次由 AI Agent 引发的生产事故

Amazon 2026 年 AI Agent 故障事件

▸ 2026 年 3 月， Amazon 电商站点连续出现严重宕机

– 3 月 5 日：订单量骤降约 99%，估计损失 630 万笔订单

– 3 月 2 日：另一起事件导致 120 万次报错、 12 万笔订单丢失

▸ 根因指向工程师采纳了 AI Agent 基于过时内部文档给出的“自信但错误”的建议

▸ 同期 AWS 内部也曾因 Kiro 智能编码工具的自主改动引发长达 13 小时的服务中断

▸ Amazon 随后启动 335 个核心系统的 90 天代码安全整改，要求高级工程师对 AI 辅助代码进行强制复核

问题的本质

▸ 现有研究多聚焦自然语言的“事实性幻觉”

▸ “代码幻觉”——生成代码看似正确，实则存在逻辑漏洞 / 依赖过时或不存在的 API——检测研究仍属空白

▸ 传统 CI/CD 依赖人工 Code Review + 规则化 Lint，难以覆盖 LLM 特有的错误模式

▸ 核心矛盾：检测覆盖率 vs. 流水线执行速度

题目九核心要求与三大创新点

核心任务

开发一个轻量级代码验证器，集成到 CI/CD 流水线中：在 AI 生成代码被合并前，
结合静态分析 + 动态沙箱测试，检测潜在的运行时错误与 API 兼容性问题，
并在检测覆盖率与执行效率之间取得平衡。

创新点一

代码幻觉分类体系

首次系统性定义 LLM 生成代码中的幻觉类型： API 误用、逻辑短路、资源泄漏、安全漏洞等

创新点二

静态 - 动态混合验证

AST 模式匹配（快） + 轻量级沙箱执行（准）相结合，实现高效的代码审查

创新点三

CI/CD 原生集成

以结构化反馈形式将验证结果返回给开发者，支持一键修正建议

文献地图：围绕三条主线收集 8 篇文献

① 幻觉的定义与分类

- ▸ CodeHalu (AAAI'25)
- ▸ Refusing the Impossible (2026)
- ▸ LLM Hallucination Survey (2025)

② 检测方法：静态 / 动态

- ▸ Static Analysis for Library Hallucinations (2026)
- ▸ Static-Pass Dynamic-Fail Gap (2026)
- ▸ Sampling-based Consensus Verification (ASE'26)

③ 风险与工程落地

- ▸ Package Hallucinations / Slopsquatting (USENIX Sec'25)
- ▸ SandboxEval (2025)

三条主线分别回答：幻觉“是什么” → “怎么发现” → “现实风险有多大 / 怎么安全落地”

分类 · 基准

CodeHalu：基于执行结果的代码幻觉分类与检测

Tian et al., AAAI 2025 · arXiv:2405.00253

- 提出四类代码幻觉： Mapping （需求 - 代码映射错误）、 Naming （命名 /API 误用）、 Resource （资源管理错误）、 Logic （逻辑短路）
- 构建 CodeHaluEval 基准： 8,883 个样本，覆盖 699 个任务
- 评测 17 个主流 LLM ，发现不同模型在幻觉率上存在显著差异
- 检测方式：执行结果驱动（ execution-based verification ），而非仅看代码表面

启示 为本项目的“幻觉分类体系”提供了第一套可直接借鉴的四分类标准。

分类 · 基准

Refusing the Impossible： 三维度幻觉分类 + 对抗基准

Dasu, Kundu, Tan · arXiv:2609.03267 (2026)

▸从三个维度定义幻觉： Groundedness （通用性违规 vs. 生态特定虚构）、 Manifestation Level （语法 / 语义 / 事实层面）、 Behavioral Pattern

▸构造 270 条“不可能任务”对抗性 prompt （6 种语言、 24 个子类别），

配 91 条可解对照组，避免模型“过度拒绝”被误判

▸测试 12 个开源模型、 4,332 条响应：约 60% 的不可能任务被“自信地”

生成了看似合理但错误的代码，仅 27% 被正确拒绝

启示 揭示模型“不会拒绝不可能的任务”，提示检测器需要覆盖“过度自信生成”这一模式。

静态检测 · 局限

静态分析的能力边界：库幻觉检测实证研究

Miranda-Pena et al. · arXiv:2604.07755 (2026)

- LLM 生成代码中引用不存在的库 / 接口的比例高达 8.1%–40%
- 纯静态分析工具能检出 16%–70% 的错误、 14%–85% 的库幻觉，

  检出率随模型和数据集大幅波动
- 人工分析给出理论上限：静态方法最多能解决 48.5%–77% 的问题
- 结论：静态分析是“性价比高的部分方案”，但无法独立解决问题

启示 直接支撑题目“为什么需要混合验证”——静态分析快但覆盖率有硬上限。

动态检测 · 沙箱

静态通过 ≠ 运行时安全： Static-Pass Dynamic-Fail 现象

Pourleyli, Das Urmi, Melo · arXiv:2609.10762 (2026)

▸ 三阶段流水线：静态扫描（ Bandit + Semgrep ）→ LLM 驱动的 CWE 推理
→ Docker 隔离沙箱中的自动化漏洞验证

▸ 1,355 个 Python 样本中， 654 个通过了静态扫描

▸ 在这些“静态通过”的样本中，流水线仍找出 394 个候选漏洞，

其中 95 个被确认可被实际利用

▸ 结论：静态通过与运行时安全是两个层级的保证，不可互相替代

启示 为题目的“动态沙箱测试”环节提供了可复用的三阶段流水线架构范式。

沙箱与供应链

沙箱执行安全 与 依赖投毒风险

SandboxEval (arXiv:2504.00018) · Package Hallucinations (USENIX Sec 2025)

▸ SandboxEval：针对“执行不可信 LLM 生成代码”场景设计安全测试集，
覆盖敏感信息泄露、文件系统篡改、非法外部通信等风险点

▸ Package Hallucinations：测试 16 个代码模型，包幻觉平均发生率 19.6%

（开源模型约 21%，商业模型约 5%），发现 20.5 万 + 个虚构包名

▸ 约 45% 的虚构包名在重复查询中持续出现——

攻击者可预先抢注这些包名实施“ slopsquatting” 供应链攻击

启示 提示 CI/CD 集成方案必须同时保证“沙箱本身的安全边界”与“依赖来源可信”。

把切口收窄：文档 / 测试“漂移”是幻觉的一个被忽视的诱因

回到起点案例

Amazon 3 月 5 日事故的根因，官方复盘为：工程师采纳了 AI Agent 基于“过时内部 wiki” 推断出的建议——本质上就是一次“文档已经漂移、但没人（也没有工具）提醒它已经过时”的失败。

我们观察到的缺口

▸ 已收集的幻觉检测文献（①②）多在“单次生成的代码片段”粒度上判断对错，默认代码本身自洽即可，没有对照仓库中已有的模块文档 / 测试是否同步更新

▸ 静态 / 动态验证文献（③④⑤）解决的是“这段新代码本身是否正确”，

而不是“这段代码改动是否让别处的说明或用例变得过时”

▸ 换句话说：现有工作把“幻觉”当作生成时刻的问题，我们认为它也是一个

合并后的维护问题——没有同步的文档会在下一次 AI 辅助开发中制造新的

幻觉（正如 Amazon 案例）

已有的文档 / 测试自动维护工作 —— 以及它们止步的地方

RepoAgent

arXiv:2402.16667

做了什么

git pre-commit 时定位受影响对象及
双向引用，更新“最小范围”文档

止步于：函数 / 对象级 docstring ，
无模块级架构图，不涉及测试

DocAgent

arXiv:2504.08725

做了什么

Reader/Searcher/Writer/Verifier
多智能体拓扑式生成文档

止步于：从零生成文档，
不是“ diff→ 判断是否级联更新”

CASCADE

arXiv:2604.19400

做了什么

用文档生成单测、用代码生成参照
实现，双向交叉验证一致性

止步于：单文件 / 单函数级一致性
检测，无仓库级传播机制

Repo Intelligence Graph

arXiv:2601.10112

做了什么

从构建系统抽取确定性架构图，
喂给编码 agent 提升单次任务表现

止步于：架构图只服务单次编码任务，不驱动文档 / 测试维护流程

架构感知的模块级文档 / 测试同步 Agent

Step 0 · 离线

抽取仓库架构

解析目录结构 /import 关系，聚类出组件，用 LLM 为每个组件生成一份“组件说明文档”
（职责、接口、依赖关系）

Step 1

diff → 模块归属

给定一次 PR/commit 的 diff，
依据架构图把改动文件
映射到其所属的一个或多个组件

Step 2

模块级 agent 更新

为每个受影响组件启动一个 agent，读取该组件已有说明文档 + diff，判断 / 更新文档，并更新或新增对应测试

Step 3

级联传播决策

agent 依据架构图中的依赖边，
查看相邻组件的说明文档，
自主判断这些组件是否也
需要同样处理（设深度上限）

关键设计取舍

Step 3 的“是否传播”由 agent 结合两端组件文档自主判断，而非单纯按图遍历全部邻居——避免更新无限扩散，同时保留处理跨模块真实影响的能力。

创新点：据我们检索，尚未有工作同时做到这四件事

维度	RepoAgent / DocAgent	CASCADE	Repo Intelligence Graph	我们的方法
分析粒度	函数 / 对象级	函数 / 文件级	可构建组件级（服务单次编码）	**模块 / 组件级架构图（服务维护流程）**
触发方式	commit 时局部对象引用扫描	对已有代码库批量扫描	构建系统解析，单次任务加载	**diff → 架构图 → 精确定位受影响组件**
产出	仅文档	仅一致性报告	仅架构图	**文档 + 测试同步更新**
跨模块传播	无（仅双向引用）	无	无	**agent 依据架构图自主判断是否级联**

简单研究规划：四个阶段，约 8 周

阶段一 · 第 1-2 周	阶段二 · 第 3-4 周	阶段三 · 第 5-6 周	阶段四 · 第 7-8 周
架构抽取	**单模块更新 agent**	**级联传播机制**	**CI 集成与评测**
解析目录 /import 结构聚类组件， LLM 生成组件说明文档； 在 2-3 个开源仓库上人工核对模块划分是否合理	实现 diff→ 模块归属映射； 构建“读文档 +diff→ 更新文档与测试”的单模块 agent， 用人工构造的 diff 做初步验证	基于架构依赖图设计传播判断； 从历史 PR 中挖掘“多模块同时 改文档”的样本作小型评测集， 评估级联判断的准确率	封装为 GitHub Action； 对比 agent 产出与人工真实更新（文档相似度 + 测试是否通过 / 覆盖预期行为）， 汇报延迟与成本

第一次汇报重点：完成阶段一的架构抽取原型 + 阶段二的方法设计（不要求完整实现）

参考文献列表

- [1] Tian et al. CodeHalu: Investigating Code Hallucinations in LLMs via Execution-based Verification. AAAI 2025. arXiv:2405.00253
- [2] Dasu, Kundu, Tan. Refusing the Impossible: A Taxonomy and Benchmark for Code Hallucination in LLMs. arXiv:2609.03267
- [3] Miranda-Pena et al. An Empirical Analysis of Static Analysis Methods for Detection and Mitigation of Code Library Hallucinations. arXiv:2604.07755
- [4] Pourleyli, Das Urmi, Melo. Beyond Static Guarantees: Measuring the Static-Pass Dynamic-Fail Gap in Security-Sensitive and LLM-Generated Python Code. arXiv:2609.10762
- [5] Rabin et al. SandboxEval: Towards Securing Test Environment for Untrusted Code. arXiv:2504.00018
- [6] Large Language Models Hallucination: A Comprehensive Survey. arXiv:2510.06265
- [7] Spracklen et al. We Have a Package for You! Package Hallucinations by Code Generating LLMs. USENIX Security 2025
- [8] Hallucination detection in LLM code generation: A sampling-based consensus verification approach. ASE, Springer, 2026
- [9] Luo et al. RepoAgent: An LLM-Powered Framework for Repository-level Code Documentation Generation. arXiv:2402.16667
- [10] DocAgent: A Multi-Agent System for Automated Code Documentation Generation. arXiv:2504.08725
- [11] Kiecker et al. CASCADE: Detecting Inconsistencies between Code and Documentation with Automatic Test Generation. arXiv:2604.19400
- [12] Cherny-Shahar, Yehudai. Repository Intelligence Graph: Deterministic Architectural Map for LLM Code Assistants. arXiv:2601.10112